AlgoBlitz 3.0 - Editorial

Question 1 : **Bad Day ??**

Problem Statement : A playful monitoring robot named **Bytey** checks whether a server is happy or upset based on its response number S. **Bytey** follows a simple rule: If the response code lies between 200 and 299 (inclusive), the server is happy and working perfectly. Otherwise, the server is having a bad day. Help **Bytey** decide the server's mood. Print "Success" if the server is happy; otherwise, print "Failure".

Intuition :

The problem is simply checking whether a number Slies within a specific range.

Bytey considers the server "happy" only if:

$$200 \leq S \leq 299$$

So instead of doing anything complex, we just need to verify if Sfalls inside this interval.
If yes → "Success"
Otherwise → "Failure"

Solution approach :

1. Input the integer S
2. Check the condition:
    - If $200 \leq S \leq 299$:
        - Print "Success"
    - Else:
        - Print "Failure"

This is a direct **range-checking problem** using conditional statements.

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
   int s;
   cin>>s;
   if(200<=s && s<=299)
   {
      cout<<"Success"<<endl;
   }
   else
   {
      cout<<"Failure"<<endl;
   }
}
```

Time Complexity (TC) : O(1)

Space Complexity (SC) : O(1)

Question 2 : **Fair Split**

Problem Statement: Three friends bring snacks represented by integers A, B, and C.

They want to divide the snacks into two or more groups such that every group has the same total amount. Your task is to determine whether such a fair division is possible.

Print "Yes" if the numbers can be divided into groups having equal sums; otherwise, print "No".

Intuition :

You need to split A, B, C into ≥ 2 groups such that:

- Every group has the same total sum

Since there are only 3 numbers, think about all possible ways to form groups:

You can't split a single number further, so groups must be formed by combining these numbers.

Possible groupings:

- {A} and {B, C}
- {B} and {A, C}
- {C} and {A, B}

For the sums to be equal:

- One number must exactly equal the sum of the other two

Solution approach :

Read inputs A, B, C

Check the following:

- A + B == C
- A + C == B
- B + C == A

If **any one condition is true → print "Yes"**

Otherwise → print "No"

Code :

```
#include <bits/stdc++.h>

using namespace std;

int main() {

   int A, B, C;

   cin >> A >> B >> C;

 if (A + B == C || A + C == B || B + C == A)
{

      cout << "Yes"<<endl;
}

else
{

      cout << "No"<<endl;
}

   return 0;

}
```

Time Complexity (TC) : **O(1)**

Space Complexity (SC) : **O(1)**

Question 3: **Fahhhh!**

Problem Statement : **Hrishikesh** wrote a two-digit number (n) on a board. His friend asks him to split the number and add its digits. Help Hrishikesh find the result.

Given a two-digit integer (**n**), print the **sum** of its digits.

Intuition :

A two-digit number is made up of its digits (for example, 47 → 4 and 7).
To find the sum of its digits, we need a way to **extract each digit one by one**.

We can do this using:

- **Modulo (%)** → gives last digit
- **Division (/)** → removes last digit

By repeating this, we can process all digits and keep adding them.

Solution approach :

Read number of test cases t

For each test case:

- Input number n
- Initialize sum = 0
- While n > 0:
    - Extract last digit → temp = n % 10
    - Add to sum → sum += temp
    - Remove last digit → n = n / 10

Print the sum

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main(){
 int t,n,temp,sum;
  cin>>t;
  while(t--)
 {
     cin>>n;
     sum = 0;
     while(n!=0){
          temp = n%10;
          sum = sum + temp;
           n = n/10;
    }
     cout<<sum<<endl;
     }
}
```

Time Complexity (TC) : O(log n) per test case

Space Complexity (SC) : O(1)

Question 4 : **Bob Won!!!**

Problem Statement : There are two friends, **Bob** and **John**. You have (**n**) candies and want to distribute all of them between the two such that both receive at least one candy, and Bob gets strictly more candies than John. Each distribution must use all candies, and the number of candies given to each person must be an integer. Your task is to determine the number of ways to distribute the candies under these conditions. Formally, find the number of ways to represent (**n** = **a** + **b**), where (**a**) and (**b**) are positive integers and (**a** > **b**).

Intuition :

We need to split $n$into two positive integers $a$and $b$such that:

- $a + b = n$
- $a > b$

Instead of trying all pairs, observe a pattern:

- For every valid pair, $b$must be **strictly less than half of** n
- Because if $b \geq \frac{n}{2}$, then $a \leq b$(invalid)

So we just count how many values of $b$satisfy: $1 \leq b < \frac{n}{2}$

Solution approach :

For each test case

Input n

If $n$is even:

- Valid values of b: 1to $\frac{n}{2} - 1$
- Count = $\frac{n}{2} - 1$

If $n$is odd:

- Valid values of b: 1to $\left\lfloor \frac{n}{2} \right\rfloor$
- Count = $\frac{n}{2}$

Print result

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
   int t,n;
   cin>>t;
   while(t--)
   {
      cin>>n;
      if(n%2==0)
      {
         cout<<(n/2)-1<<endl;
      }
      else
      {
         cout<<(n/2)<<endl;
      }
   }
}
```

Time Complexity (TC) : **O(1)** per test case

Space Complexity (SC) : **O(1)**

Question 5 : Prince and the Mysterious Number

Problem Statement : **Prince** found a mysterious number (**N**) written on a note. He recalls that some numbers are called prime they have no divisors other than 1 and themselves. Curious to classify the number, he decides to check whether it belongs to this special group.

Given an integer (**N**), determine whether it is **prime** or **not**.

Intuition :

A prime number is a number greater than 1 that has only two divisors: 1 and itself.
So instead of checking all numbers up to N, we can check if any number from 2 to sqrt(N) divides N.
If we find even one divisor → not prime. Otherwise → prime.

Solution approach :

Input integer N

If $N \leq 1$, print "NO" (not prime)

Loop from $i = 2$ to $i \leq sqrt(N)$

- If $N\%i == 0$:
    - Print "NO" and stop

If no divisor is found → print "YES"

Code :

```
#include <bits/stdc++.h>

using namespace std;

int main()

{

  long long int N,i,flag = 0;

  cin>>N;

  if(N<=1)

  {

    cout<<"NO"<<endl;

    return 0;

  }

  for(i=2;i<=sqrt(N);i++)

  {

    if(N%i==0)
```

```
        {
            flag = 1;
            break;
        }
    }
    if(flag==1)
        {
            cout<<"NO"<<endl;
        }
        else
        {
            cout<<"YES"<<endl;
        }
    return 0;
}
```

Time Complexity (TC) : O(√N)

Space Complexity (SC) : O(1)

Question 6 : The Missing Number Mystery

Problem Statement : A math detective is investigating a suspicious sequence.

You are given two integers A and B. Your task is to find how many integers x exist such that A, B, and x can be rearranged to form an arithmetic sequence.

A sequence of three integers p,q,r is an arithmetic sequence if:

q−p=r−q

In other words, the difference between consecutive elements must be the same. Count how many possible values of x satisfy this condition.

It can be proven that the answer is always finite.

Intuition :

We already know there are 3 possible values of x:

- x = 2A - B (extend backward)
- x = 2B - A (extend forward)
- x = (A + B) / 2 (middle, only if even)

Case 1: A == B

if(A == B)

All formulas give same value:

2A - B = A
2B - A = A
(A+B)/2 = A

Only **one unique x**

Output = 1

Case 2: (A + B) % 2 == 0

else if((A+B)%2==0)

- Middle value exists
- Forward and backward also exist
- All three are **different**

Output = 3

Case 3: Otherwise

else

- Middle value is **not integer**
- Only forward and backward valid

Output = 2

Solution approach :

You have A and B, and you want a number x so that all three make a proper pattern (same gap).

There are only 3 ways x can fit:

- Before A
- Between A and B
- After B

What to check

1. If A == B

- All positions give the same x
  Only 1 answer

---

2. If (A + B) is even

- You can put x in the middle also

  Total 3 answers

---

3. Otherwise

- Middle is not possible
  Only 2 answers

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
   int A,B;
   cin>>A>>B;
   if(A == B)
   {
      cout<<1<<endl;
   }
   else if((A+B)%2==0)
   {
      cout<<3<<endl;
   }
   else
   {
      cout<<2<<endl;
   }
}
```

Time Complexity (TC) : **O(1)**

Space Complexity (SC) : **O(1)**

Question 7 : The Color-Coded Chaos

Problem Statement : Designer "**Pruthviraj**" is very picky about how he decorates rooms. He just lined up N colorful vases in a row. The vases are either Red (R), Green (G), or Blue (B).He has a very strict rule: No two vases next to each other can have the same color. If he sees two Red vases sitting side-by-side, he loses his mind! To keep him happy, you need to remove some vases from the row so that every vase is a different color from its neighbor. Find the minimum number of vases you need to take away to make him happy.

Intuition :

Pruthviraj wants no two adjacent vases to have the same color.

So whenever we see:

- Two consecutive characters that are equal (like "RR" or "GG")

One of them must be removed. To minimize removals, we only remove when a violation happens. So the problem reduces to: Count how many times adjacent characters are the same

Solution approach :

Input N and string S

Initialize count = 0

Loop from i = 0 to n-2:

- If S[i] == S[i+1]:
  - Increment count (one vase must be removed)

Print count

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
   int N,count = 0,i;
   string S;

   cin>>N>>S;

   for(i=0;i<N-1;i++)
   {
      if(S[i] == S[i+1])
      {
         count++;
      }
   }
   cout<<count<<endl;
}
```

Time Complexity (TC) : **O(N)**

Space Complexity (SC) : **O(N)**

Question 8 : Dhurandhar 3????

Problem Statement : In the heart of Pakistan, the legendary Indian Secret agent **Hamza Ali Mazari** known for his sharp instincts and unmatched intelligence receives a coded array (**a**) of (**n**) positive integers. It is known that in this array, all the numbers except **one** are the same (for example, in the array **[4,11,4,4]** all numbers except one are equal to **4**). At first glance, everything appears uniform, but Hamza quickly senses that **one** element is different a hidden anomaly in an otherwise perfect pattern. With time running out, he must identify the **exact position** of this suspicious element. The array is **indexed** starting from one. Help Hamza uncover the impostor by printing the index of the element that does not match the others.

Intuition

- In the array:
  - All elements are same
  - Only one is different
- So there are only 2 values total
  - One appears many times (majority)
  - One appears once (unique)

goal: find the position of the unique one

Solution approach : For each test case –

1. Take input array v
2. Make a copy a
3. Sort the copy a

After sorting:

- The majority element will occupy most positions
- The unique element will be either:
  - at the beginning OR
  - at the end

so, if(a[1] != v[i])

- a[1] is the majority element (since at least 2 elements are same)
- Now compare each v[i]:
  - The one not equal to a[1] → is the unique element

Print its index

Code :

```
#include <bits/stdc++.h>
using namespace std;
int main(){
   int t,n,i;
   cin>>t;
   while(t--){
      cin>>n;
      vector<int>v(n);
      vector<int>a(n);
      for(i=0;i<n;i++){
         cin>>v[i];
         a[i] = v[i];
      }
      sort(a.begin(),a.end());
      for(i=0;i<n;i++){
         if(a[1] != v[i]){
            cout<<i+1<<endl;
            break;
         }
      }
   }
}
```

Time Complexity (TC) : O(nlogn) per test case

Space Complexity (SC) : O(n)